

NBACore v8.3.9

Basketball Ontology Layer

AI Development Work Contract

Version

v8.3.9

Module Type

Core Data Intelligence Infrastructure

Execution Target

WorkBuddy / Codex / Local AI Coding Agent

1. Module Objective

建立 NBACore 篮球数据语义层。

目标：

让不同来源、不同命名、不同规则环境的数据能够映射到统一篮球概念。

2. Problem Definition

现实篮球数据存在大量名称差异。

例如：

NBA：

PTS

FIBA：

Points

Excel:

Score

中文数据:

得分

实际上:

它们代表:

Basketball Concept:

Points

NBACore 不应该要求所有数据源改变格式。

而应该:

理解不同数据。

3. Core Principle

Ontology Layer 不负责计算。

它负责:

External Data

|

Semantic Mapping

|

Basketball Concept

|

Analytics Engine

4. Architecture Position

当前:

Import
↓
Data Asset
↓
Formula Engine
↓
Visualization

调整:

Import
↓
Ontology Layer
↓
Provenance Layer
↓
Data Asset
↓
Analytics Engine

5. Core Objects

Ontology 系统包含：

5.1 Basketball Concept

定义篮球领域概念。

例如：

```
{
  "id": "points",
  "name": "Points",
  "category": "scoring",
  "description": "Total points scored"
}
```

5.2 External Field Mapping

外部字段映射。

例如：

输入：

PTS

映射：

points

数据：

```
{
  "source_field": "PTS",
  "target_concept": "points",
}
```

```
"confidence":0.99
}
```

5.3 Metric Definition

定义高级指标。

例如：

True Shooting Percentage

```
TS%
=
PTS/(2*(FGA+0.44FTA))
```

6. Ontology Classification

篮球概念分类：

Scoring

得分相关：

```
points
field_goals
three_points
free_throws
```

Rebounding

```
rebounds
offensive_rebounds
```

defensive_rebounds

Playmaking

assists

turnovers

potential_assists

Defense

steals

blocks

deflections

fouls

Efficiency

TS%

eFG%

PER

BPM

7. Data Model

7.1 Concepts Table

```
CREATE TABLE basketball_concepts(
```

```
id INTEGER PRIMARY KEY,
```

```
name TEXT,  
  
category TEXT,  
  
description TEXT,  
  
created_at DATETIME  
  
);
```

7.2 Field Mapping Table

```
CREATE TABLE field_mappings(  
  
id INTEGER PRIMARY KEY,  
  
source_name TEXT,  
  
source_field TEXT,  
  
concept_id INTEGER,  
  
confidence FLOAT  
  
);
```

7.3 Metric Definition Table

```
CREATE TABLE metric_definitions(  
  
id INTEGER PRIMARY KEY,  
  
name TEXT,  
  
formula TEXT,  
  
category TEXT,  
  
version TEXT  
  
);
```

8. Backend Structure

创建：

```
backend/  
  
services/  
  
ontology/  
    concept_manager.py  
    mapper.py  
    metric_registry.py  
    dictionary.py
```

9. Core Classes

OntologyManager

```
class OntologyManager:  
  
    def create_concept(self):  
        pass  
  
    def map_field(self):  
        pass  
  
    def search_concept(self):  
        pass
```

FieldMapper

```
class FieldMapper:

    def match(
        source_field
    ):

        pass
```

10. Mapping Logic

匹配优先级：

Level 1

Exact Match

例如：

```
PTS
↓
points
```

Level 2

Dictionary Match

例如：

```
Score
↓
points
```

Level 3

AI Assisted Match

例如：

得分

↓

points

11. Dictionary System

创建：

ontology_dictionary.json

示例：

```
{
  "PTS":
    "points",

  "Score":
    "points",

  "REB":
    "rebounds",

  "AST":
    "assists"
}
```

12. Formula Engine Integration

Formula Engine 不直接读取：

PTS

而读取：

points

例如：

用户公式：

$\text{points} * 0.3 + \text{rebounds} * 0.2$

这样：

NBA：

PTS

FIBA：

Points

都可以运行。

13. Multi League Support

Ontology 必须支持：

NBA

OREB

DREB

REB

FIBA

Offensive Rebound

Defensive Rebound

Total Rebound

NCAA

ORB

DRB

TRB

最终：

offensive_rebounds

14. Rules Difference Handling

注意：

Ontology 不消除规则差异。

例如：

NBA：

Defensive Three Seconds

FIBA：

不存在。

Ontology 保存:

```
{
  "concept": "defensive_3_seconds",
  "availability":
  {
    "NBA": true,
    "FIBA": false
  }
}
```

15. API Requirements

Search Concept

GET:

```
/api/ontology/search?name=PTS
```

返回:

```
{
  "concept": "points",
  "confidence": 0.99
}
```

Create Mapping

POST:

```
/api/ontology/mapping
```

Request:

```
{  
  "source_field": "score",  
  
  "concept": "points"  
}
```

16. Frontend Requirements

第一阶段：

提供：

Mapping Interface

显示：

External Field

|

Suggested Concept

|

Confirm

例如：

Score

↓

Points

[Confirm]

17. Testing Requirements

Unit Tests

测试：

PTS映射points

REB映射rebounds

自定义字段创建

Integration Test

流程：

Excel：

Score

Board

Pass

↓

Ontology Mapping

↓

Formula Engine

↓

Analysis

18. Acceptance Criteria

完成标准：

系统可以：

☒ 理解不同数据字段

✓ 创建标准篮球概念

✓ 支持多联赛数据

✓ 支持用户自定义字段

✓ 不修改原始数据

19. Development Notes For AI Agent

重要原则：

不要：

- 删除原始字段
- 强制转换用户数据
- 假设所有规则相同

必须：

保存：

Original Data

+

Semantic Meaning

该模块是未来：

- Multi League
- Formula Marketplace
- AI Analyst
- Player Intelligence

的基础。

END